

Lecture 17: Training Transformers

Week 5, Lecture 3 - From Architecture to Implementation

PSYC 51.07: Models of Language and Communication

Winter 2026

Today's Agenda

1. 📌 **Positional Encoding:** Injecting sequence order
2. 🔄 **Feed-Forward Networks:** The other key component
3. 🔗 **Layer Norm & Residuals:** Training deep networks
4. ⚡ **FlashAttention:** Making transformers faster
5. 🏗️ **Three Architectures:** Encoder, Decoder, Both
6. 💻 **Practical Implementation:** Using HuggingFace

Goal: Complete understanding of transformer training and implementation

Positional Encoding

Problem: Self-attention is permutation-invariant!

Without position information:

- "The cat sat" = "sat cat the" = "cat the sat"
- Order matters in language!

Solution: Add positional encodings to input embeddings

Original Transformer (Sinusoidal):

```
1# PE(pos, 2i) = sin(pos / 10000^(2i/d))
2# PE(pos, 2i+1) = cos(pos / 10000^(2i/d))
3
4# Position 0, dim 0: sin(0/10000^0) = 0
5# Position 1, dim 0: sin(1/10000^0) = 0.84
6# Position 2, dim 0: sin(2/10000^0) = 0.91
```

Why Sinusoidal Positional Encoding?

Advantages of sine/cosine functions:

1. Unique Patterns

- Each position gets unique vector
- Different frequencies per dimension

2. Relative Position Info

- $PE(pos+k) = \text{linear function of } PE(pos)$
- Model learns relative positions

3. Extrapolation

- Works for sequences longer than training
- No need to retrain

Visualizing Positional Encodings

Each position gets a unique pattern across dimensions

```
1Pos 0 -> Pos 5 -> Pos 9 -> Dim 0 -> Dim 4 -> Dim 7
```

Key Insight:

- Lower dimensions: Rapid oscillation (high frequency)
- Higher dimensions: Slow oscillation (low frequency)
- Creates a unique "barcode" for each position
- Model learns to use these patterns for positional awareness

Feed-Forward Networks

After attention, apply position-wise feed-forward network

Architecture: Two linear layers with ReLU/GELU activation

```

1 class FeedForward(nn.Module):
2     def __init__(self, d_model=768, d_ff=3072): # 4x expansion
3         super().__init__()
4         self.linear1 = nn.Linear(d_model, d_ff) # 768 → 3072
5         self.linear2 = nn.Linear(d_ff, d_model) # 3072 → 768
6         self.relu = nn.ReLU()
7
8     def forward(self, x):
9         # x: [batch, seq_len, 768]
10        x = self.linear1(x) # [batch, seq_len, 3072]
11        x = self.relu(x) # Non-linearity!
12        x = self.linear2(x) # [batch, seq_len, 768]
13        return x
14

```

Why Feed-Forward Networks? 🤔

Role in the Transformer:

1. Add Non-linearity

- Attention is mostly linear operations (weighted sums)
- FFN introduces non-linear transformations
- ReLU/GELU activation adds expressiveness

2. Position-wise Processing

- Attention mixes information across positions
- FFN processes each position independently
- Allows position-specific feature transformations

3. Increase Model Capacity

- Expansion (4x) provides more parameters

Layer Normalization & Residual Connections

Critical for training deep transformers!

Residual Connections:

```
1# output = sublayer(x) + x
2x = x + self.attention(x)
3x = x + self.feedforward(x)
```

- Gradients flow directly through
- Prevents vanishing gradients
- Enables 96-layer models (GPT-3)!

Layer Normalization:

Pre-Norm vs Post-Norm

Two ways to arrange LayerNorm and residual connections

Post-Norm (Original):

```
1# Normalize AFTER residual  
2x = LayerNorm(x + Attention(x))  
3x = LayerNorm(x + FFN(x))
```

- Used in original Transformer, BERT
- Can be unstable for deep models
- Requires careful initialization

Pre-Norm (Modern):

Complete Transformer Block

Putting it all together:

```

1 class TransformerBlock(nn.Module):
2     def __init__(self, d_model=768, n_heads=12, d_ff=3072):
3         super().__init__()
4         self.attention = MultiHeadAttention(d_model, n_heads)
5         self.ffn = FeedForward(d_model, d_ff)
6         self.norm1 = nn.LayerNorm(d_model)
7         self.norm2 = nn.LayerNorm(d_model)
8
9     def forward(self, x):
10        # Self-attention with residual
11        x = x + self.attention(self.norm1(x))
12        # Feed-forward with residual
13        x = x + self.ffn(self.norm2(x))
14        return x

```

FlashAttention: Making Transformers Faster ⚡

Problem: Standard attention is slow and memory-hungry!

Standard Attention Complexity

- Time: $O(n^2)$ where n = sequence length
- Memory: $O(n^2)$ to store attention matrix
- Bottleneck: Reading/writing to GPU memory (HBM)

FlashAttention Innovation:

- Tile-based computation
- Uses fast SRAM instead of slow HBM
- Fused operations (fewer memory reads)
- Recomputation in backward pass

Other Attention Optimizations

Addressing the $O(n^2)$ problem:

1. Sparse Attention

- Only attend to subset of positions
- Local windows + global tokens
- Used in: Longformer, BigBird

2. Linear Attention

- Approximate attention with linear complexity
- Kernel trick to avoid materializing attention matrix
- Used in: Performer, Linear Transformer

3. Low-Rank Approximation

- Factorize attention matrix

Three Transformer Architectures

Architecture	How it works	Examples \	Uses
Best for: Classification, NER, QA			
Best for: Generation, completion			
Decoder: masked + cross-attention	T5, BART, mT5		
Best for: Translation, summarization			

Key Difference: Attention Masking

- **Encoder:** Full self-attention (bidirectional)
- **Decoder:** Causal/masked attention (unidirectional)

Visual Comparison: Encoder vs Decoder vs Both

```
1**Encoder-Only (BERT) -> Self-Attention -> bidirectional -> \textbf{Decoder-Only}
```

```
\end{center}
```

Choosing the Right Architecture:

- Need to understand full context? → **Encoder** (BERT)
- Need to generate text? → **Decoder** (GPT)
- Need both (translate, summarize)? → **Encoder-Decoder** (T5)

Using Transformers in Practice

HuggingFace makes it easy!

```
1from transformers import AutoModel, AutoTokenizer
2
3# Load pre-trained model (downloads ~440MB first time)
4model_name = "bert-base-uncased"
5tokenizer = AutoTokenizer.from_pretrained(model_name)
6model = AutoModel.from_pretrained(model_name)
7
8# Tokenize input
9text = "The animal didn't cross the street because it was too tired"
10inputs = tokenizer(text, return_tensors="pt")
11print(inputs['input_ids'])
12# tensor([[ 101, 1996, 4111, 2134, 1005, 1056, 2892, 1996,
13#          2395, 2138, 2009, 2001, 2205, 5458, 102]])
14#      [CLS] The animal didn ' t cross the ...
15
16# Get contextualized embeddings
```

Using Transformers in Practice

```
21it_embedding = hidden_states[0, 10, :] # 768-dim context-aware vector
```

...continued

Reference: HuggingFace Course - Chapter 1.4

Comparing Architectures: Code Examples

Different architectures for different tasks

```
1 from transformers import AutoModelForSequenceClassification, \
2                           AutoModelForCausalLM, \
3                           AutoModelForSeq2SeqLM
4
5 # 1. ENCODER-ONLY (BERT): Classification
6 encoder_model = AutoModelForSequenceClassification.from_pretrained(
7     "bert-base-uncased", num_labels=2
8 )
9 # Use for: Sentiment analysis, NER, classification
10
11 # 2. DECODER-ONLY (GPT): Text generation
12 decoder_model = AutoModelForCausalLM.from_pretrained("gpt2")
13 # Use for: Text completion, creative writing, few-shot learning
14
15 # 3. ENCODER-DECODER (T5): Seq2Seq tasks
16 seq2seq_model = AutoModelForSeq2SeqLM.from_pretrained("t5-base")
```

Practical Tips for Training Transformers

1. Learning Rate & Warmup

```
1# Warmup: gradually increase LR
2# Then decay (linear or cosine)
3scheduler = get_linear_schedule_with_warmup(
4    optimizer,
5    num_warmup_steps=1000, # ~10% of training
6    num_training_steps=10000
7)
8# Fine-tuning: lr=2e-5, From scratch: lr=1e-4
```

2. Optimizer

```
1optimizer = AdamW(
2    model.parameters(),
3    lr=2e-5,
```

Computational Efficiency Tips

1. Batch Size

- Larger batches = better GPU utilization
- Use gradient accumulation if GPU memory limited
- Typical: effective batch size 256-2048 tokens

2. Sequence Length

- Shorter sequences train faster (quadratic complexity!)
- Consider truncation or sliding windows
- Pack multiple examples to maximize GPU usage

3. Model Size

- Start small, scale up if needed

- DistilBERT: 40% smaller, 60% faster, 97% performance

Discussion Questions

1. Positional Encoding:

- Why add instead of concatenate?
- What happens without positional encoding?
- Learned vs. fixed: which is better?

2. Architecture Choice:

- When would you use encoder-only vs decoder-only?
- Can GPT do classification? Can BERT generate?
- Why has decoder-only become more popular recently?

3. Scaling:

- Is bigger always better?
- What are the limits to scaling transformers?

Looking Ahead to Week 6

This week (Week 5) we learned:

- Attention mechanisms (Lecture 12)
- Self-attention and transformer architecture (Lecture 13)
- Training transformers: all the components (Lecture 14)

Next week (Week 6):

- Masked Language Modeling
- Pre-training and fine-tuning
- Contextual embeddings in action

\item

- RoBERTa, ALBERT, DistilBERT

Summary

Key Takeaways:

1. Positional Encoding

- Sine/cosine functions inject position information
- Enables model to understand order

2. Feed-Forward Networks

- Position-wise transformations
- Add non-linearity and capacity

3. LayerNorm & Residuals

- Critical for training deep networks
- Stabilize gradients, enable deeper models

References

Essential Papers:

- **Vaswani et al. (2017)** - "Attention Is All You Need"
- The original Transformer paper
- **Su et al. (2021)** - "RoFormer: Enhanced Transformer with Rotary Position Embedding"
 - Modern positional encoding approach
- **Dao et al. (2022)** - "FlashAttention: Fast and Memory-Efficient Exact Attention"
 - Making transformers faster

Tutorials:

- HuggingFace Course: Chapters 1.4, 1.5

Questions?

Discussion Time

Topics for discussion:

- Positional encoding approaches
- Architecture choices
- Training tips and tricks
- Implementation questions
- Assignment 4 preparation

Thank you! 

See you next week for BERT!